AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

LECTURE 02

Mathematical Foundations: Gradients, Curvature, Convexity

Week 2

Dr. Taha Yiğit Alkan · Department of Computer Engineering

[Fall / 2026 - 2027] · CSE 413

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

Three Parts, One Habit: Compute It

PART I · CALCULUS & LINEAR ALGEBRA

- Vectors, inner products, three norms
- Gradient: geometry + a cookbook
- Jacobian, chain rule, Hessian
- Taylor models (1-D and 2-D, with errors)
- Quadratic forms · definiteness tests

PART II · CONVEXITY

- Convex sets and what preserves them
- Convex functions: Jensen with numbers
- A gallery: convex / concave / neither
- First- and second-order tests
- The payoff: local = global (measured)

PART III · NUMERICAL REALITY

- Floating point: $0.1 + 0.2 \neq 0.3$
- Catastrophic cancellation, and the fix
- Vectorization: ~1,000× for free
- Finite differences: the V-curve
- Conditioning: when answers jump

Four Objects Power Every Algorithm

Last week: how to pose a problem. This week: the machinery every solver uses to answer it.

The gradient ∇f

Which way is downhill — and how steep?

drives gradient descent (Wk 5) and SGD (Wk 10)

The Hessian $\nabla^2 f$

How does the terrain curve?

drives Newton's method (Wk 6); decides min vs saddle

The Taylor model

A simple local stand-in for f.

what every solver actually minimizes at each step

Convexity

Is a local minimum THE minimum?

separates solvable (Wks 3–9) from hard (Wks 10–14)

Vectors & Inner Products: Alignment as a Number

- **A decision vector x (n numbers) is a POINT — and also a DIRECTION to move in.**
- Inner product $x^Ty = x_1y_1 + \dots + x_ny_n$ measures alignment.
- $x^Ty = \|x\| \|y\| \cos\theta$ — so it encodes the ANGLE between directions.
- $x^Ty = 0 \Leftrightarrow$ perpendicular. $x^Ty > 0$: roughly the same way. < 0: opposing.
- **Why optimizers care: a step p is a descent direction exactly when $\nabla f^Tp < 0$.**

Cauchy–Schwarz: $|x^Ty| \le \|x\| \|y\|$. It is why cos θ stays in [−1, 1] — and why the steepest-descent direction (slide 8) is exactly $-\nabla f$.

WORKED EXAMPLE

x = (3, 4), y = (4, −3), z = (6, 8), e_1 = (1, 0)

$x^Ty = 3\cdot4 + 4\cdot(-3) = 0 \rightarrow$ perpendicular

$x^Tz = 18 + 32 = 50 = \|x\|\|z\| = 5\cdot10 \rightarrow$ parallel ($\cos\theta = 1$)

$x^Te_1 = 3$, $\cos\theta = 3 / (5\cdot1) = 0.6 \rightarrow \theta = 53.13°$

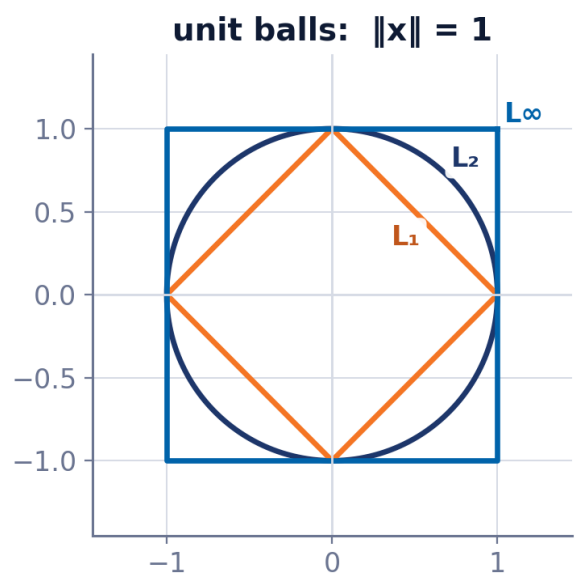
projection of x onto e_1: $(x^Te_1)\, e_1 = (3, 0)$

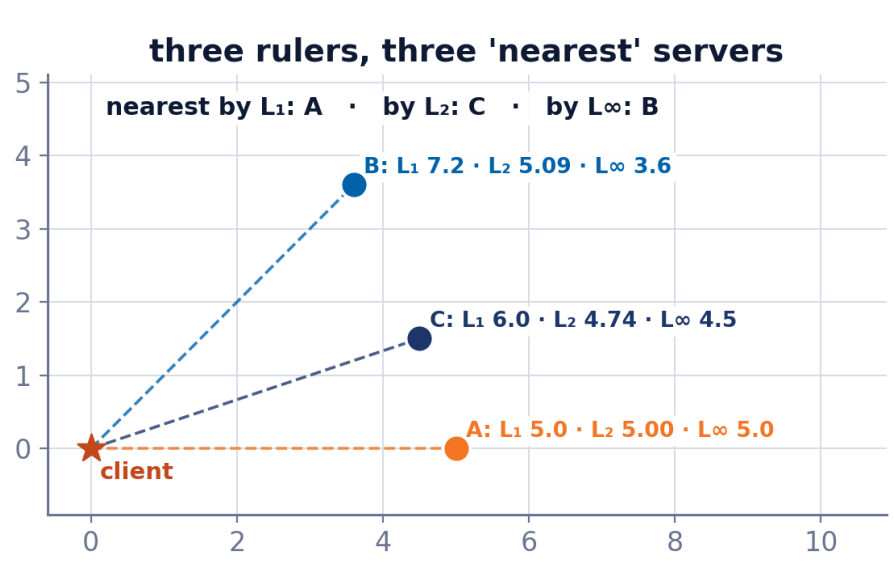
Descent check: $\nabla f = (4, 9)$, $p = (-1, 0)$:

$\nabla f^Tp = -4 < 0 \rightarrow$ p is a descent direction ✓

$p = (9, -4)$: $\nabla f^Tp = 36 - 36 = 0 \rightarrow$ f unchanged (1st order)

Three Norms: Same Point, Different Rulers





WORKED EXAMPLE

v = (3, –4)

$\|v\|_1 = |3| + |-4| = 7$

$\|v\|_2 = \sqrt{(9 + 16)} = 5$

$\|v\|_\infty = \max(3, 4) = 4$

always: $\|v\|_\infty \le \|v\|_2 \le \|v\|_1$

here: $4 \le 5 \le 7$ ✓

The 'nearest server' depends on the ruler. Client at the origin; servers A (5, 0), B (3.6, 3.6), C (4.5, 1.5). L_1 (sum of hop costs) picks A. L_2 (straight-line) picks C. L_∞ (worst single coordinate) picks B. Three norms, three different answers — choosing the norm IS choosing the objective (Week 1's lesson, already back).

Which Ruler for Which Job?

NORM	WHAT IT MEASURES	WHERE YOU WILL MEET IT
L_2 (Euclidean)	sum of squares — smooth, differentiable, energy-like	**least squares, ridge regression, gradient step length (Wks 5, 9)**
L_1 (Manhattan)	sum of $\lvert\cdot\rvert$ — corners on the axes, kink at zero	**sparsity: Lasso, model compression (Wks 9–11)**
$L\infty$ (max)	the single worst entry	**worst-case error, minimax design, Chebyshev fitting (Wk 8)**

WORKED EXAMPLE

: errors e = (1, 1, 1, 1, 10) vs e′ = (3, 3, 3, 3, 3)

$\|e\|_1 = 14 < \|e'\|_1 = 15 \quad \|e\|_2 = 10.2 > \|e'\|_2 = 6.7 \quad \|e\|_\infty = 10 > \|e'\|_\infty = 3 \quad \rightarrow$ L_1 forgives the outlier; L_2 and $L\infty$ punish it

The Gradient: All First-Order Information, One Vector

$$\nabla f(x) = \left(\partial f/\partial x_1,\ \partial f/\partial x_2,\ \ldots,\ \partial f/\partial x_n \right)^T$$

differentiate with respect to each variable, holding the others fixed; stack the results

- **Direction: ∇f points where f increases fastest; −∇f is steepest DESCENT.**
- Length: ‖∇f‖ is the slope in that direction — the local steepness.
- Geometry: ∇f is perpendicular to the contour (level set) through the point.
- **Optimality: at an interior minimum, ∇f(x*) = 0 — a 'stationary point'.**
- Caution: ∇f = 0 is necessary, NOT sufficient — slide 12 shows a trap.

WORKED EXAMPLE

f(x, y) = x² + xy + 2y² at the point P = (1, 2)

$\partial f/\partial x = 2x + y \qquad \partial f/\partial y = x + 4y$

$\nabla f(x, y) = (\ 2x + y\ ,\ x + 4y\ )$

$f(P) = 1 + 2 + 8 = 11$

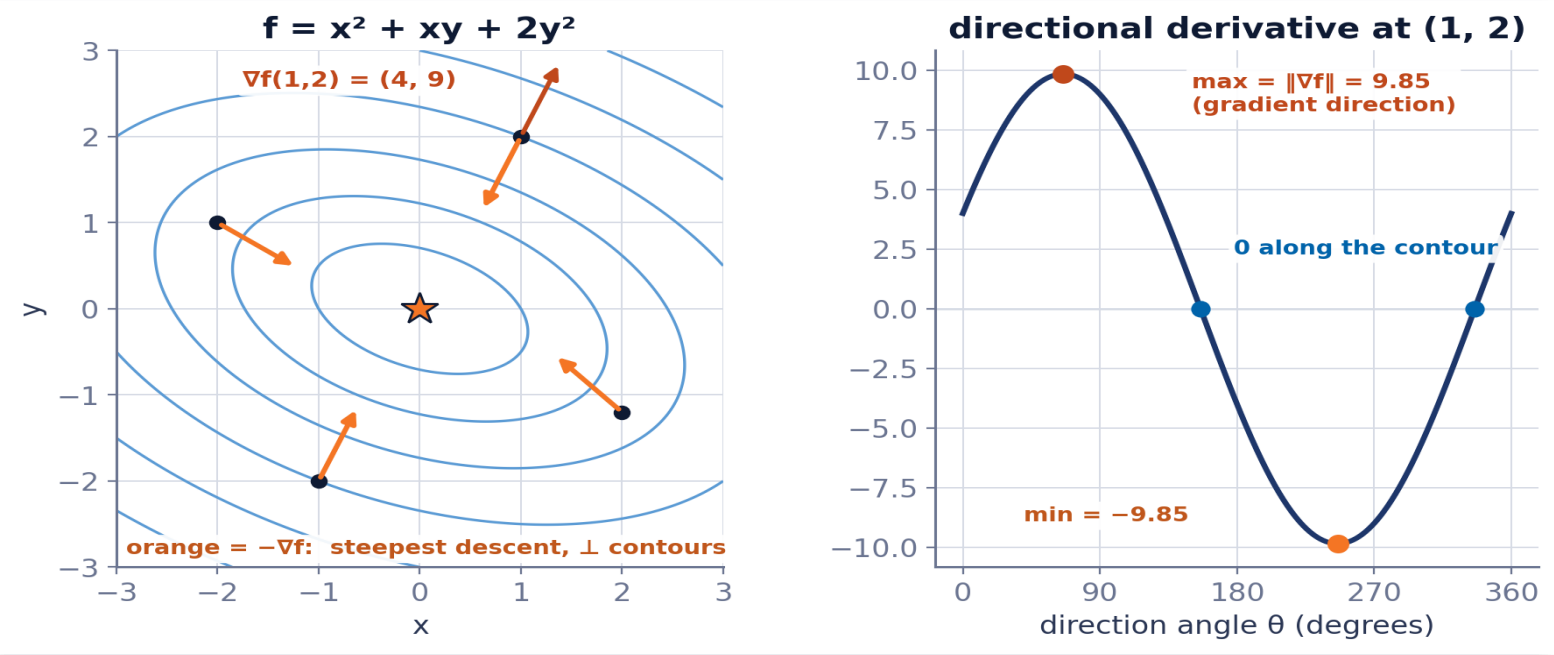
$\nabla f(P) = (\ 2\cdot 1 + 2\ ,\ 1 + 4\cdot 2\ ) = (4, 9)$

$\|\nabla f(P)\| = \sqrt{(16 + 81)} = \sqrt{97} \approx 9.85$

Stationary point: $2x + y = 0,\ x + 4y = 0$

$\rightarrow\ (x, y) = (0, 0)$ and $f(0, 0) = 0$

Gradient Geometry: Read the Map, Pick a Direction



Directional derivative = $\nabla f^T u = \|\nabla f\| \cos\theta$. A cosine wave over directions: maximal along the gradient, zero along the contour, most negative straight against it. Every descent method is a rule for choosing u in the negative half of that wave.

WORKED EXAMPLE

slope at (1, 2) along unit u: $\nabla f^T u$

$u = (1, 0)$: 4

$u = (0, 1)$: 9

$u = \nabla f/\|\nabla f\|$: +9.85 (max)

$u = (-9, 4)/\sqrt{97}$: 0 (contour)

$u = -\nabla f/\|\nabla f\|$: −9.85 (min)

all values lie between

$-\|\nabla f\|$ and $+\|\nabla f\|$ (Cauchy–Schwarz)

A Gradient Cookbook (You Will Use These All Term)

FUNCTION	GRADIENT	WHERE IT RETURNS
$\mathbf{f(x) = a^T x + b}$	$\nabla f = a$	linear objective — LP (Wk 3)
$\mathbf{f(x) = x^T A x}$	$\nabla f = (A + A^T)x$ [= 2Ax if A symmetric]	quadratic forms (slide 15)
$\mathbf{f(x) = \tfrac{1}{2} x^T Q x - b^T x}$	$\nabla f = Qx - b$	every quadratic model (slide 17)
$\mathbf{f(x) = \|Ax - b\|^2}$	$\nabla f = 2A^T(Ax - b)$	least squares (Wk 5)
$\mathbf{f(x) = \log \Sigma \exp(x_i)}$	$\nabla f = \text{softmax}(x)$	classification losses (Wk 11)

WORKED EXAMPLE

Least squares: fit a line through (1,1), (2,2), (3,2) → A = [[1,1],[1,2],[1,3]], b = (1, 2, 2)

at $x = (0, 0)$: $\nabla f = 2A^T(0 - b) = -2A^T b = (-10, -22)$ — far from optimal, steep

set $\nabla f = 0$: $A^T A\, x = A^T b$ → $[[3,6],[6,14]]\, x = (5, 11)$ → $x^* = (2/3, 1/2)$: the line $y = 0.667 + 0.5t$

Jacobian & Chain Rule: Derivatives of Pipelines

- **For a vector map F (n inputs → m outputs) the Jacobian J is the m×n matrix of all first partials.**
- Row i = gradient of output i. A scalar function's Jacobian is its gradient (as a row).
- J is the best LINEAR approximation: $F(x + p) \approx F(x) + J\,p$.
- **Chain rule: for a pipeline h = f(F(x)), $\nabla h = J^T \nabla f$. Multiply Jacobians, right to left.**

Why a computer engineer cares: a neural network is a long pipeline of maps. Backpropagation (Week 11) is NOTHING but this chain rule, applied layer by layer from the output backwards — J^T times the gradient, again and again.

WORKED EXAMPLE

$F(x, y) = (x^2y,\ 5x + \sin y),\quad f(u, v) = u + v^2$

$J(x, y) = [[\,2xy,\ x^2\,],\ [\,5,\ \cos y\,]]$

at $(1, 0)$: $J = [[0, 1], [5, 1]]$, $F(1, 0) = (0, 5)$

$\nabla f(u, v) = (1, 2v) \rightarrow$ at $(0, 5)$: $(1, 10)$

$\nabla h = J^T \nabla f = [[0, 5], [1, 1]]\ (1, 10) = (50, 11)$

The Hessian: Curvature as a Matrix

$$\nabla^2 f(x) = [\, \partial^2 f / \partial x_i \partial x_j \,] \qquad \text{(an } n \times n \text{ matrix)}$$

symmetric, because mixed partials commute — so its eigenvalues are real

- **How the GRADIENT changes: $\nabla f(x + p) \approx \nabla f(x) + \nabla^2 f(x)\, p$.**
- Curvature along a unit direction u is the number $u^T \nabla^2 f\, u$.
- Eigenvalues = the extreme curvatures; eigenvectors = the directions where they occur.
- **All eigenvalues > 0: a bowl. Mixed signs: a saddle. That is the whole second-order test.**
- Ratio λmax/λmin = the condition number κ — it will set gradient descent's speed (Wk 5).

WORKED EXAMPLE

Two Hessians, computed

$f = x^2 + xy + 2y^2$: $f_{xx} = 2,\ f_{xy} = 1,\ f_{yy} = 4$

$\nabla^2 f = [[2, 1], [1, 4]]$ (constant — f is quadratic)

eigenvalues $3 \pm \sqrt{2} = 1.59$ and $4.41 \rightarrow$ bowl, $\kappa = 2.78$

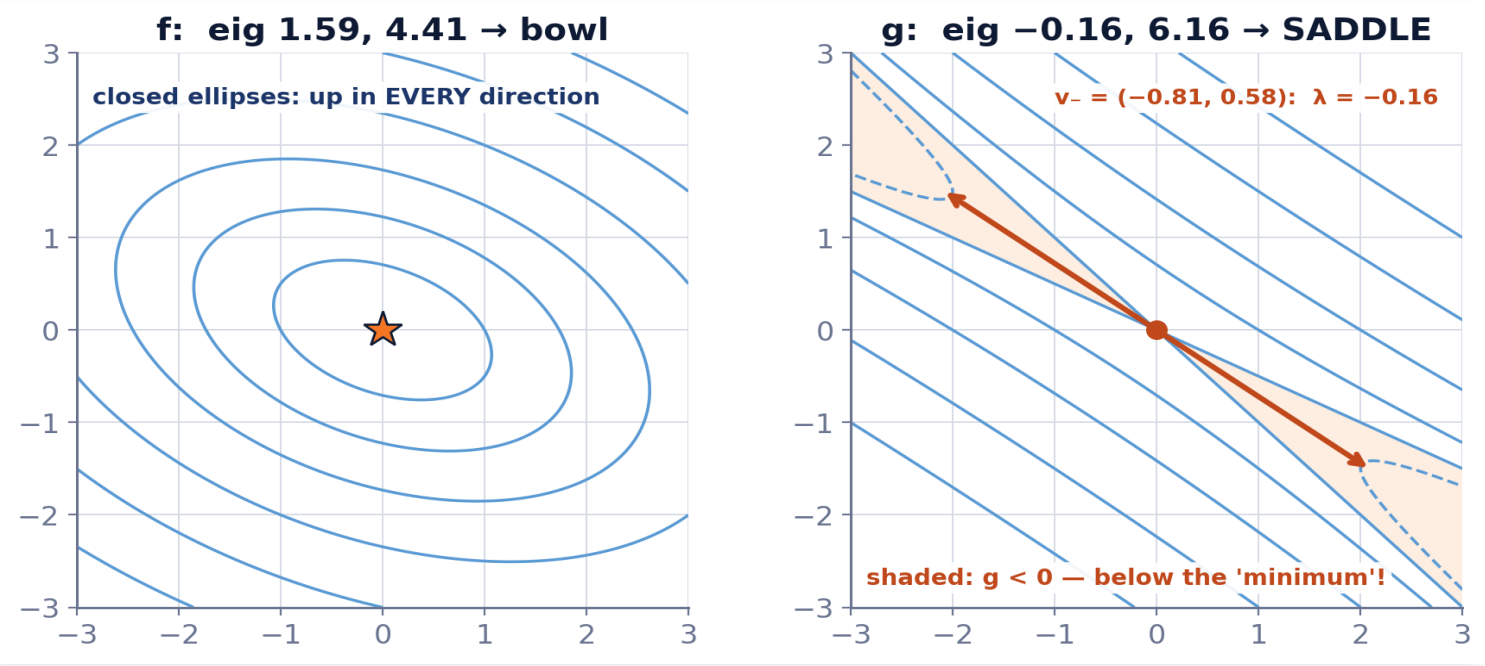
Rosenbrock $(1 - x)^2 + 100(y - x^2)^2$ at its minimum (1, 1):

$\nabla^2 f = [[802, -400], [-400, 200]]$

eigenvalues 0.40 and 1001.6 → bowl, but $\kappa \approx 2{,}508$

a canyon: 2,508× more curved across than along

A Trap: All-Positive Coefficients, and Still Not a Minimum



Never judge by the coefficients. f and g differ only in the xy term (1 vs 3), both have ∇ = 0 at the origin — but only the EIGENVALUES of the Hessian reveal that g slopes downhill in one narrow direction.

WORKED EXAMPLE

$g(x, y) = x^2 + 3xy + 2y^2$

$\nabla g = (2x + 3y, 3x + 4y) = 0$ at $(0, 0)$

$\nabla^2 g = [[2, 3], [3, 4]]$

$\det = 8 - 9 = -1 < 0 \rightarrow$ mixed signs

eigenvalues: -0.16 and 6.16

along $v_{neg} = (-0.81, 0.58)$:

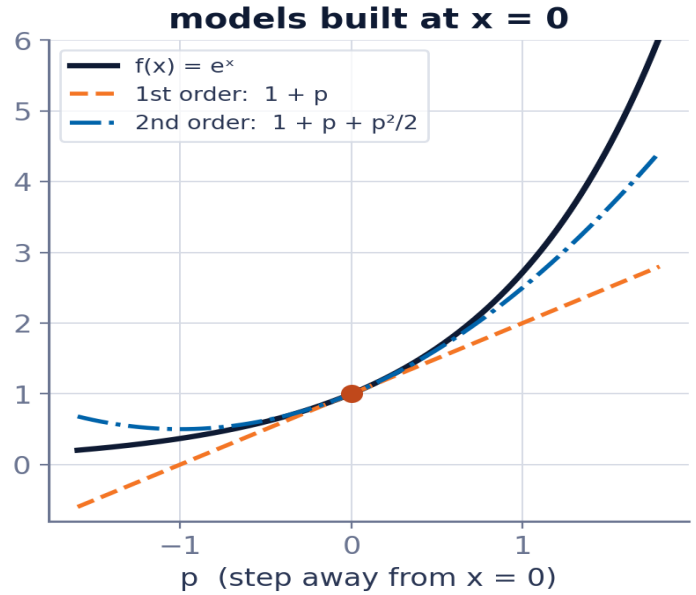
$g(2v_{neg}) = -0.32 < g(0, 0) = 0$

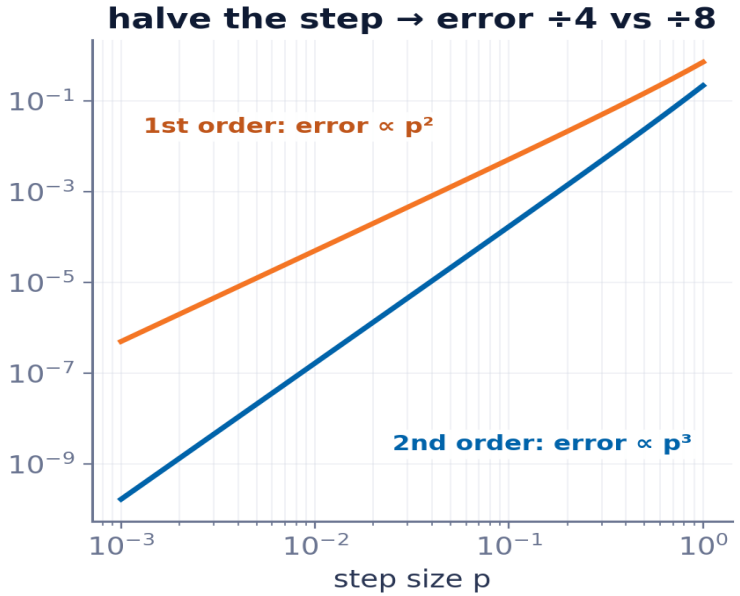
→ the origin is a SADDLE, not a min

Taylor Models: Replace f by Something You Can Minimize

$$f(x+p) \approx f(x) + \nabla f(x)^T p + \tfrac{1}{2} p^T \nabla^2 f(x)\, p$$

constant + linear (gradient) + quadratic (Hessian)





WORKED EXAMPLE

f(x) = eˣ at x = 0: errors

step p	1st order	2nd order
0.1	0.0052	0.00017
0.5	0.149	0.024
1.0	0.718	0.218

halve p: 1st-order error ÷4,

2nd-order error ÷8

Taylor in 2-D: Which Model Would You Trust with a Step?

WORKED EXAMPLE

Rosenbrock $f = (1 - x)^2 + 100(y - x^2)^2$ at $x_0 = (-1, 1)$, step $p = (0.1, 0.05)$

$f(x_0) = 4 + 0 = 4$

$\nabla f(x_0) = (-4, 0)$

$\nabla^2 f(x_0) = [[802, 400], [400, 200]]$

linear model: $4 + \nabla f^T p = 4 - 0.4 = 3.60$

quadratic model: $3.60 + \frac{1}{2}\, p^T \nabla^2 f\, p$

$= 3.60 + \frac{1}{2}(8.02 + 4.00 + 0.50) = 9.86$

TRUE value: $f(-0.9, 1.05) = 3.61 + 5.76 = 9.37$

Read the verdict

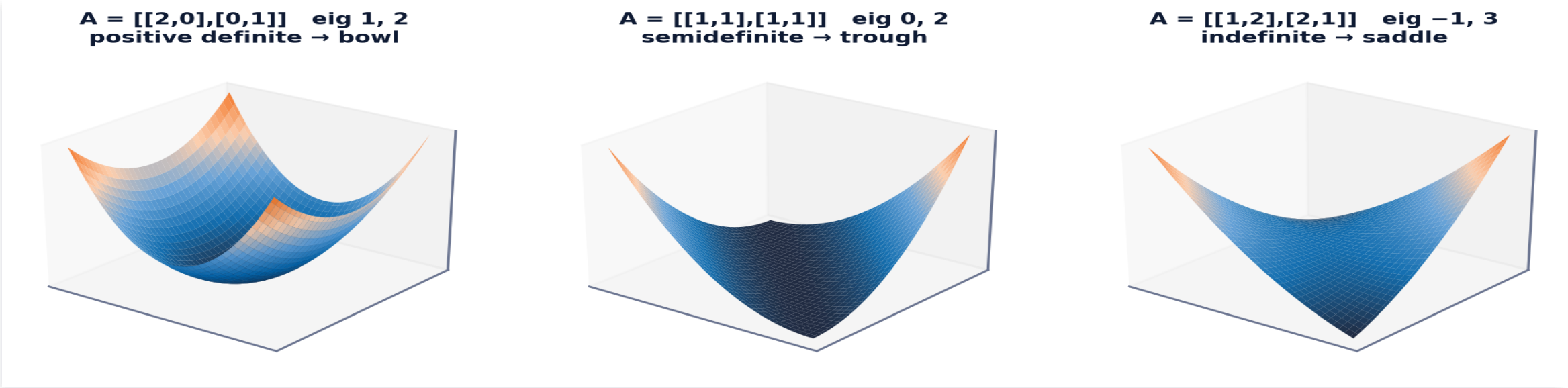
- **Linear model predicts f DROPS to 3.60. Truth: it RISES to 9.37.**
- Quadratic model says 9.86 — right direction, 5% off.
- The linear model was wrong about the SIGN of the change.

Why it matters

- In a curved canyon the slope alone lies about even a small step — curvature dominates.
- **This is Rosenbrock's trap for gradient descent (Wk 5) and Newton's advantage (Wk 6).**

Quadratic Forms: Three Matrices, Three Landscapes

$q(x) = x^T A\, x$ with A symmetric — the shape is decided entirely by the eigenvalues of A



Positive definite

$x^T Ax > 0$ for every $x \neq 0$. All eigenvalues > 0. A bowl: unique minimum.

Positive semidefinite

$x^T Ax \geq 0$. Eigenvalues ≥ 0, some = 0. A trough: flat along (1, −1) here.

Indefinite

Both signs occur. Eigenvalues of both signs. A saddle: no minimum at all.

Three Ways to Test Positive Definiteness

1 · Eigenvalues

`np.linalg.eigvalsh(A)`

Compute all eigenvalues; PD ⇔ all > 0. The definition-level test — also gives κ = λmax/λmin.

2 · Leading minors (Sylvester)

`pen and paper`

PD ⇔ every top-left k×k determinant > 0. The by-hand test for small matrices.

3 · Cholesky

`np.linalg.cholesky(A)`

Try to factor $A = LL^T$. Succeeds ⇔ PD. The fastest test — what solvers actually run.

Semidefinite fine print: for PSD, leading minors are NOT enough (all principal minors must be ≥ 0) and Cholesky may fail on the boundary — use eigenvalues with a small tolerance.

WORKED EXAMPLE

A = [[4, 1, 0], [1, 3, 1], [0, 1, 2]]

Test 2 — leading minors:

$D_1 = 4 > 0$

$D_2 = 4 \cdot 3 - 1 \cdot 1 = 11 > 0$

$D_3 = 4(3 \cdot 2 - 1) - 1(1 \cdot 2 - 0) = 20 - 2 = 18 > 0$

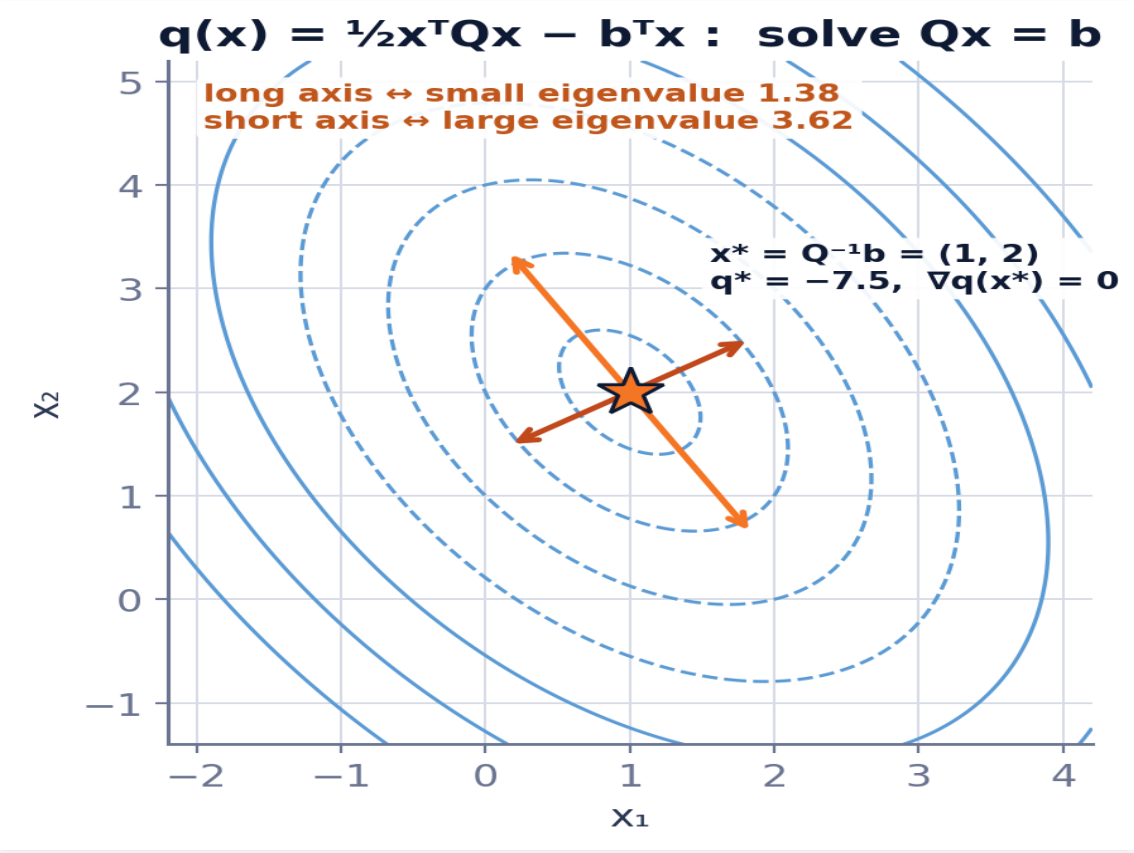
→ positive definite ✓

Test 1 — eigenvalues: 1.27, 3.00, 4.73 (all > 0 ✓)

κ = 4.73 / 1.27 = 3.7 (well conditioned)

Test 3 — Cholesky succeeds ✓ (Lab 2 runs all three)

Minimizing a Quadratic: The Problem Inside Every Other Problem



WORKED EXAMPLE

$q(x) = \tfrac{1}{2}x^TQx - b^Tx$, $Q = [[3, 1], [1, 2]]$, $b = (5, 5)$

$\nabla q = Qx - b = 0 \;\rightarrow\; \text{solve } Qx = b$

$\det Q = 5, \quad Q^{-1} = (1/5)\,[[2, -1], [-1, 3]]$

$x^* = (1/5)(10 - 5, -5 + 15) = (1, 2)$

$q^* = \tfrac{1}{2}\cdot 15 - 15 = -7.5$

$\text{eig}(Q) = 1.38, 3.62 \;\rightarrow\; \text{PD, so } x^* \text{ is THE minimum}$

Why this slide matters most in Part I: the second-order Taylor model IS such a q. Newton's method (Wk 6) = 'solve Qx = b' at every iteration; least squares (slide 9) was the same thing with $Q = A^TA$. Contours are ellipses along Q's eigenvectors, stretched by $1/\sqrt{\lambda}$ — the geometry behind κ.

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

PART II

Convexity: When Local Means Global

One structural property separates the problems we can solve from the ones we can only attack.

Convex Sets: No Dents, No Holes



- **C is convex if the segment between ANY two of its points stays in C: λx + (1−λ)y is in C for every λ between 0 and 1.**
- Convex: balls, half-spaces $a^T x \le b$, hyperplanes, polyhedra $Ax \le b$, the PSD matrices.
- **Preserved by INTERSECTION and affine maps — so any set of linear constraints is convex.**
- NOT preserved by union: two separate discs are not a convex set.

WORKED EXAMPLE

$C = \{ x : x_1 + x_2 \le 4,\ x \ge 0 \}$

take $u = (4, 0)$ and $w = (0, 4)$ in C

midpoint ($\lambda = ½$): $(2, 2)$

$2 + 2 = 4 \le 4$ ✓ $2, 2 \ge 0$ ✓ → in C

$\lambda = ¼$: $(1, 3)$: $1 + 3 = 4 \le 4$ ✓ → in C

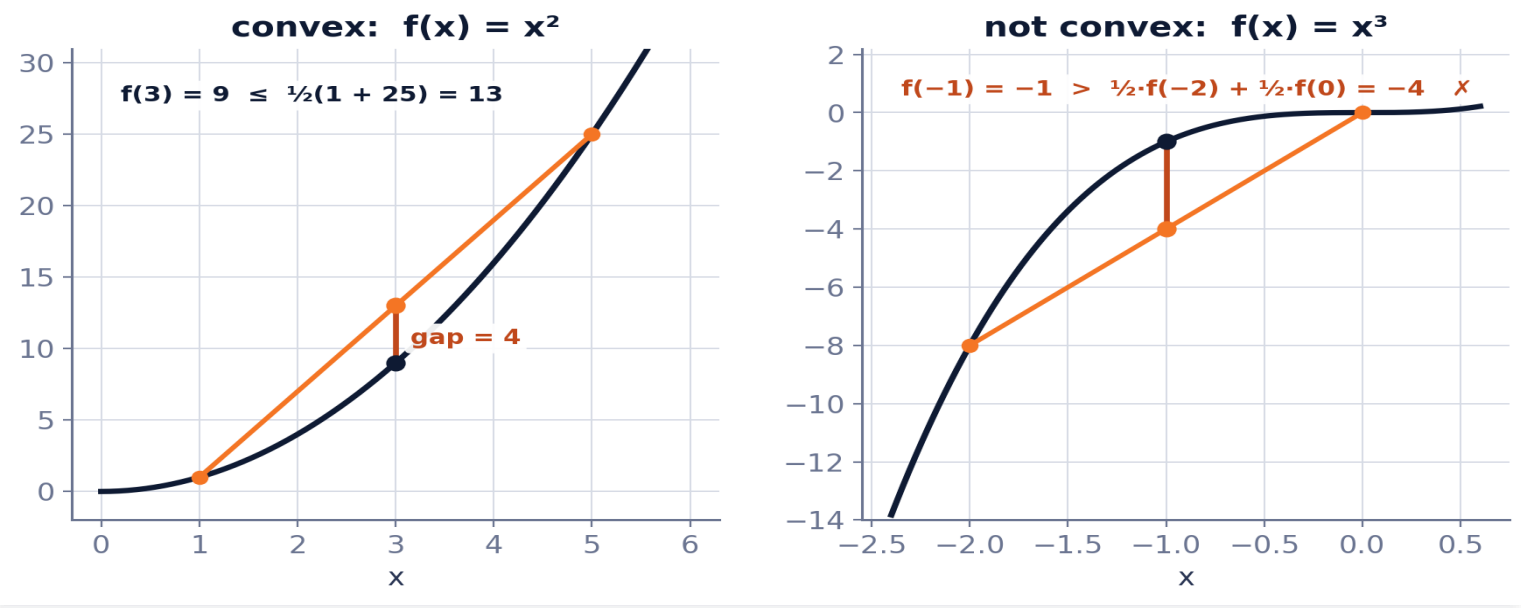
Non-example: integer points of C

$(1, 0)$ and $(2, 0)$ are in; midpoint

$(1.5, 0)$ is not → NOT convex

Convex Functions: The Chord Test, With Numbers

$$f(\lambda x + (1-\lambda)y) \le \lambda f(x) + (1-\lambda) f(y) \quad \text{for all } x, y \text{ and } \lambda \text{ in } [0, 1] \quad \text{— the graph never rises above a chord}$$



WORKED EXAMPLE

Run the test yourself

$f = x^2$, $x = 1$, $y = 5$, $\lambda = ½$:

$f(3) = 9 \le ½(1 + 25) = 13$ ✓

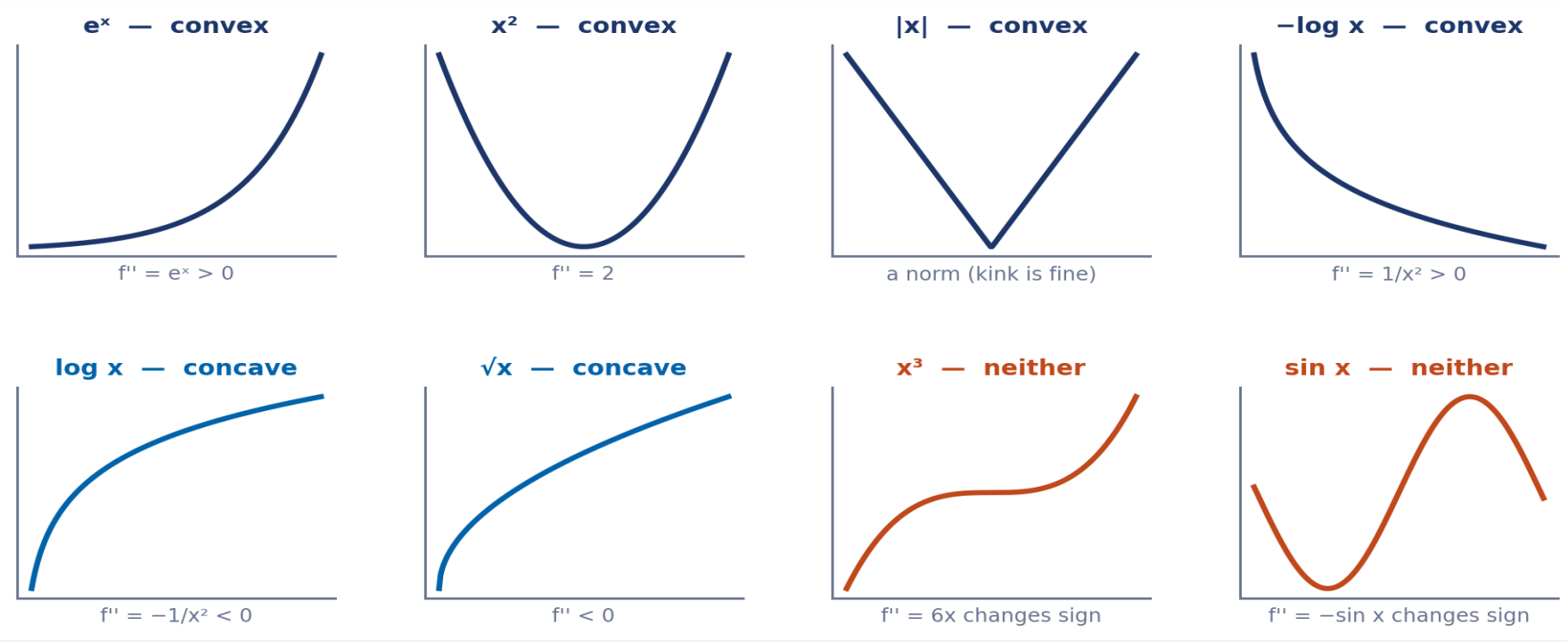
$f = x^3$, $x = -2$, $y = 0$, $\lambda = ½$:

$f(-1) = -1 > ½(-8 + 0) = -4$ ✗

ONE violated pair disproves convexity;

no number of passing pairs proves it.

A Gallery: Classify on Sight

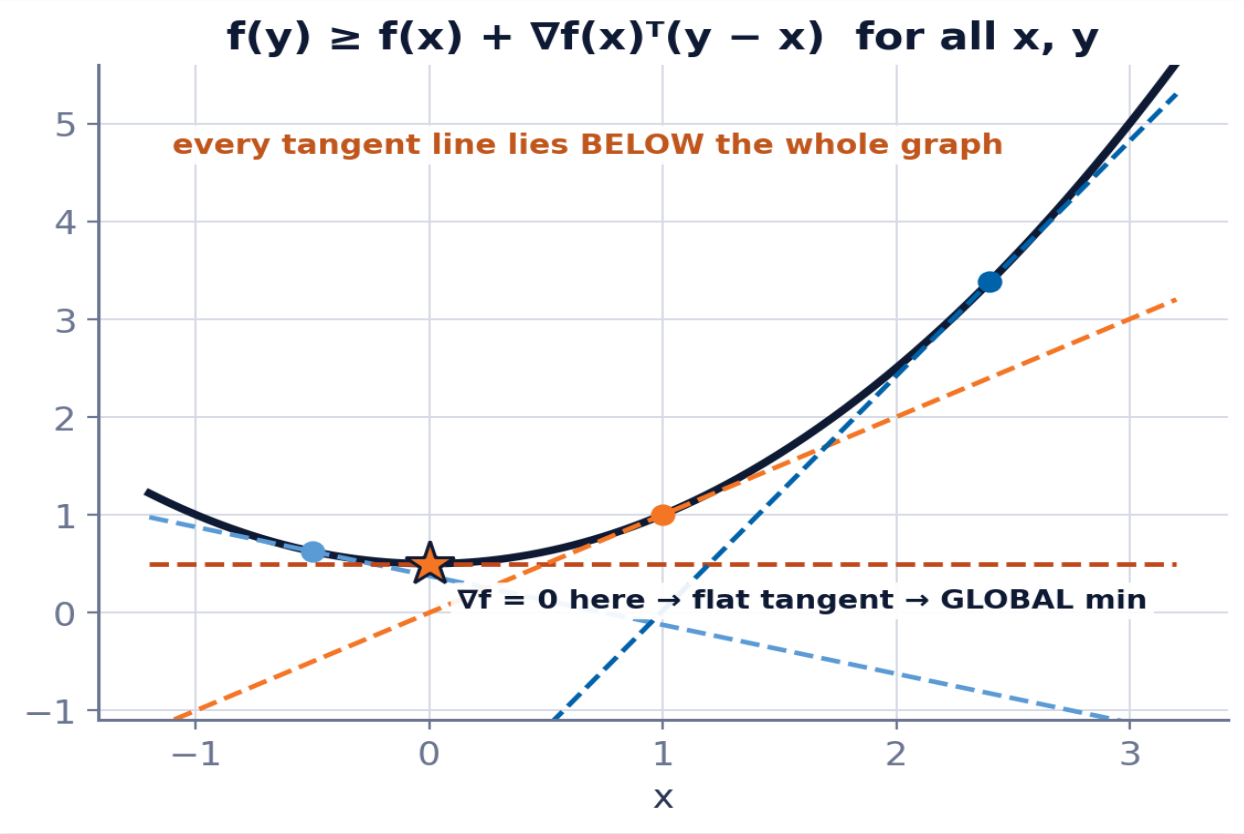


1-D rule

- **f″ ≥ 0 everywhere → convex**
- **f″ ≤ 0 everywhere → concave**
- sign change → neither
- kinks are allowed: |x| is convex
- f″ > 0 → STRICTLY convex
- a line is both convex and concave

Domain matters: 1/x is convex on x > 0 and concave on x < 0 — on the whole line it is neither. x^3 is convex on x ≥ 0. Always state WHERE. Concave = −convex: maximizing a concave function is the same easy problem.

First-Order Test: Tangents Never Overshoot



- **For differentiable f: convex ⇔ f(y) ≥ f(x) + ∇f(x)ᵀ(y − x) for all x, y.**
- The first-order Taylor model is a GLOBAL under-estimator — local information with a global guarantee.

WORKED EXAMPLE

f(x) = ½x² + ½

tangent at x = 1: $f(1) + f'(1)(y - 1) = 1 + (y - 1) = y$

check y = 3: $f(3) = 5 \geq 3$ ✓ y = −1: $f(-1) = 1 \geq -1$ ✓

at $x^* = 0$: $f'(0) = 0 \rightarrow$ tangent is the constant ½

so $f(y) \geq \frac{1}{2} = f(0)$ for EVERY y → global minimum

Second-Order Test: Three Functions, Three Verdicts

f is convex ⇔ $\nabla^2 f(x)$ is positive SEMIdefinite at every x. Positive definite everywhere ⇒ strictly convex (unique minimizer). Recipe: Hessian → eigenvalues → read the signs.

WORKED EXAMPLE

1 · $f = x^2 + xy + y^2$

$\nabla^2 f = [[2, 1], [1, 2]]$

eigenvalues: 1 and 3

both > 0 everywhere

→ STRICTLY CONVEX

unique minimizer (0, 0)

WORKED EXAMPLE

2 · $g = x^2 + 3xy + 2y^2$

$\nabla^2 g = [[2, 3], [3, 4]]$

eigenvalues: −0.16 and 6.16

a negative eigenvalue

→ NOT CONVEX

(the saddle of slide 12)

WORKED EXAMPLE

3 · $h = \log(e^x + e^y)$

$\nabla^2 h = \mathrm{diag}(p) - pp^T$, p = softmax

at (0, 0): [[¼, −¼], [−¼, ¼]]

eigenvalues: 0 and ½

→ CONVEX, not strictly

(flat along (1, 1): h shifts by t)

Certify by Construction: The Calculus of Convexity

- **Building blocks (convex): affine $a^Tx + b$ · every norm · e^x · $-\log x$ · $\frac{1}{2}x^TQx$ with Q PSD · log-sum-exp.**
- Rule 1 — nonnegative weighted sums of convex functions are convex.
- Rule 2 — composition with an affine map: f(Ax + b) is convex if f is.
- Rule 3 — the pointwise maximum of convex functions is convex.
- **No Hessians needed: recognize the blocks, apply the rules, done.**

This is how CVXPY thinks (Week 9). Its 'disciplined convex programming' checker does not compute eigenvalues — it parses your model into these blocks and rules. If you can certify a model by hand on this slide, you can predict what the solver will accept.

WORKED EXAMPLE

Certify three models by inspection

1 · Lasso: $\|Ax - b\|_2^2 + \lambda\|x\|_1 \quad (\lambda \geq 0)$

$(\text{norm})^2$ of affine, Rule 2 + λ·norm, Rule 1 → convex ✓

2 · worst-case latency: $\max(\, a_1^Tx + b_1 \,, \ldots, a_m^Tx + b_m \,)$

max of affine functions (Rule 3) → convex ✓

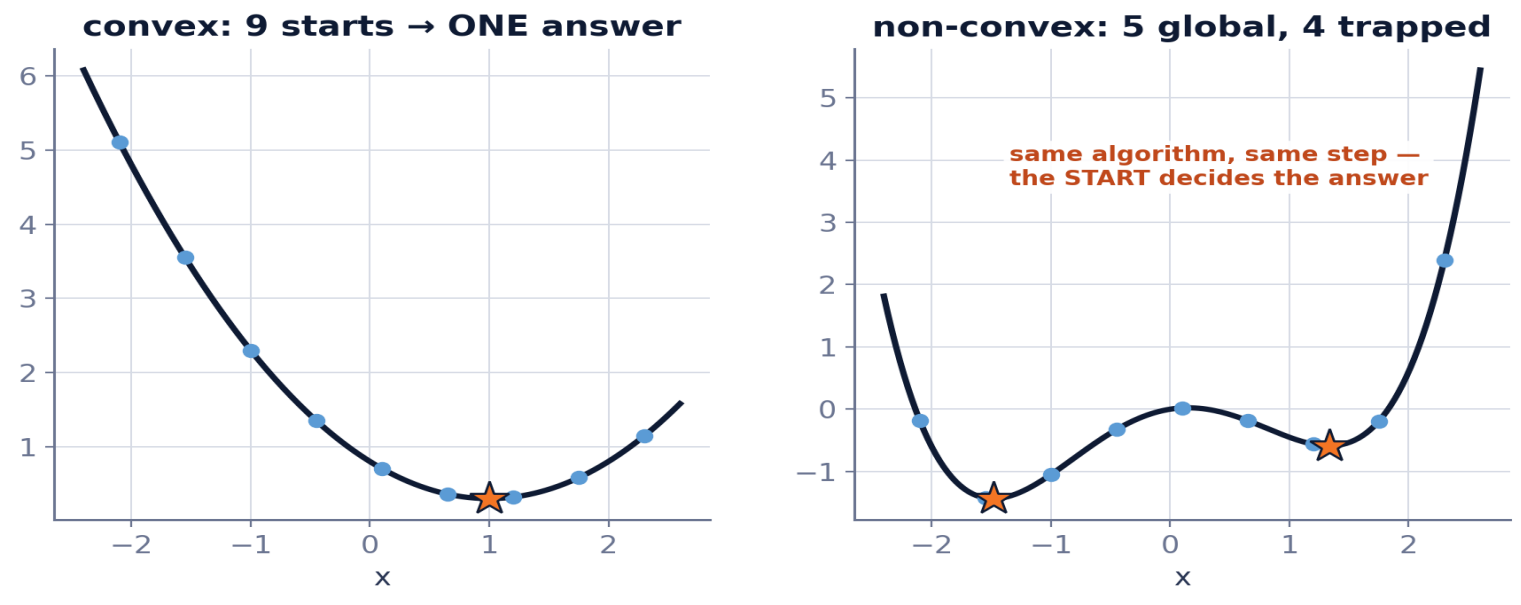
3 · $f(x, y) = x \cdot y$ — looks innocent

$\nabla^2 f = [[0, 1], [1, 0]]$, eigenvalues $-1, +1$ → NOT convex ✗

products of variables are the #1 convexity-breaker

(bilinear terms: Week 8 shows how to avoid them)

The Payoff, Measured: Nine Starts, One Answer



WORKED EXAMPLE

Same algorithm, 9 starting points

convex $\frac{1}{2}(x-1)^2 + 0.3$:

all 9 runs → x = 1.000

non-convex $x^4/4 - x^2 + 0.3x$:

5 runs → x = −1.484 (global)

4 runs → x = +1.332 (trapped)

Local = global

In a convex problem there are no spurious valleys — whatever minimum you find is THE minimum.

∇f = 0 is a certificate

Stationarity is sufficient (slide 22), so any descent method that converges has solved the problem.

Optimality is provable

Duality (Wks 4, 7, 9) turns the answer into a checkable proof — a guarantee, not a hope.

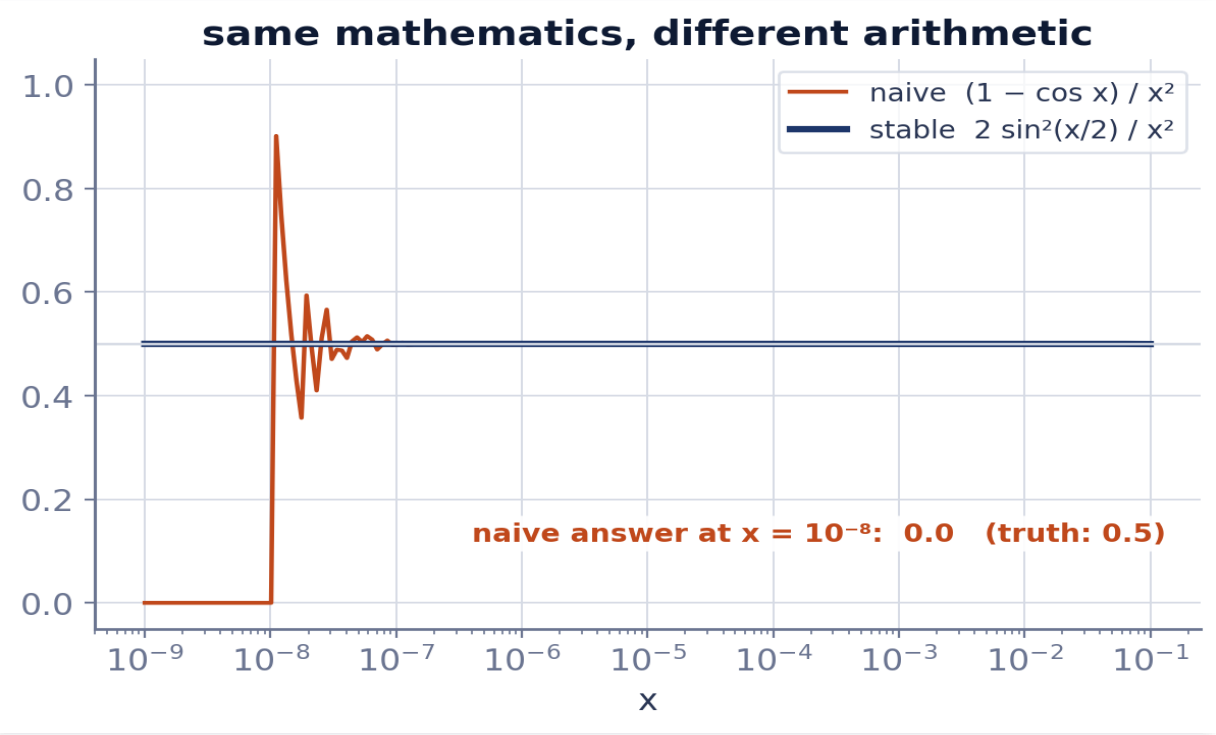
AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

PART III

Numerical Reality: When Mathematics Meets the Machine

Real numbers have infinitely many digits. Your computer has sixteen.

Floating Point: Sixteen Digits and Their Consequences



WORKED EXAMPLE

Three experiments (run them in Lab 2)

$0.1 + 0.2 \rightarrow 0.30000000000000004$

machine epsilon $\varepsilon = 2.22 \times 10^{-16}$ (spacing near 1.0)

$(1 - \cos x)/x^2$ at $x = 10^{-8}$ (truth: 0.5)

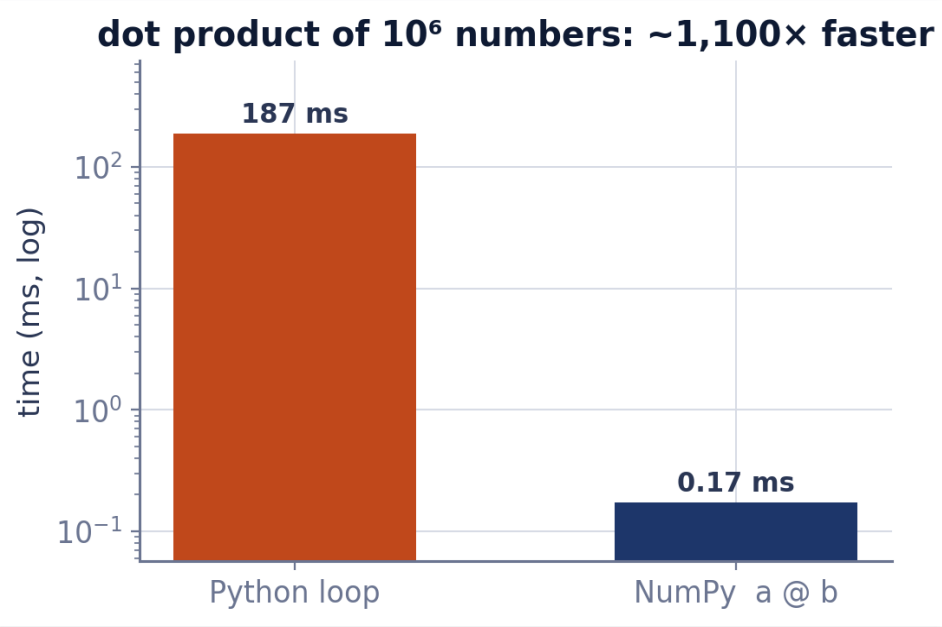
naive formula → 0.0 ← 100% error

$2\sin^2(x/2)/x^2 \rightarrow 0.5$ ← exact

$\log(e^{1000} + e^{1000})$: naive → inf ; shift trick → 1000.6931

Catastrophic cancellation: subtracting nearly equal numbers deletes the leading digits and keeps the noise. $\cos(10^{-8})$ rounds to exactly 1, so $1 - \cos x = 0$. Fix: rewrite the FORMULA (same mathematics, different arithmetic). Never compare floats with == ; use a tolerance.

Vectorization: Three Orders of Magnitude for Free



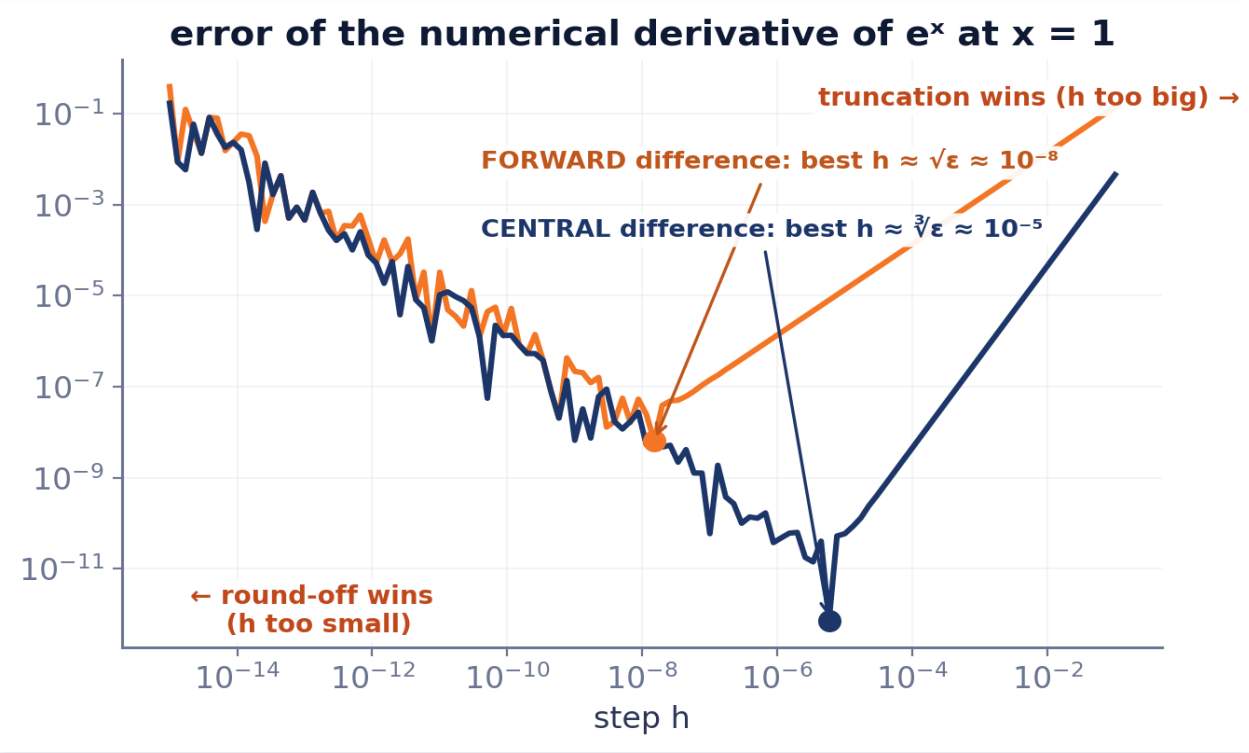

```
# slow: the interpreter executes a million tiny steps
s = 0.0
for i in range(len(a)):
    s += a[i] * b[i]              # ~190 ms

# fast: one call into compiled, SIMD-parallel BLAS
s = a @ b                         # ~0.15 ms
g = 2 * A.T @ (A @ x - b)         # the slide-9 gradient, verbatim
```

The rule for this course: if you wrote a Python loop over array elements, you wrote a bug. Express the mathematics as matrix–vector operations — the code then reads like the formula on the slide (compare the last line with the cookbook), runs ~1,000× faster (measured on this laptop: 10^6-element dot product), and is what makes the 20,000-sample experiments of Week 10 feasible at all.

AKDENİZ UNIVERSITY
FACULTY OF ENGINEERING

Finite Differences: How to Check a Gradient



WORKED EXAMPLE

Derivative of e^x at x = 1 (truth: 2.71828…)

forward: $(f(x+h) - f(x))/h$ error $\sim h$

central: $(f(x+h) - f(x-h))/2h$ error $\sim h^2$

measured best step and error:

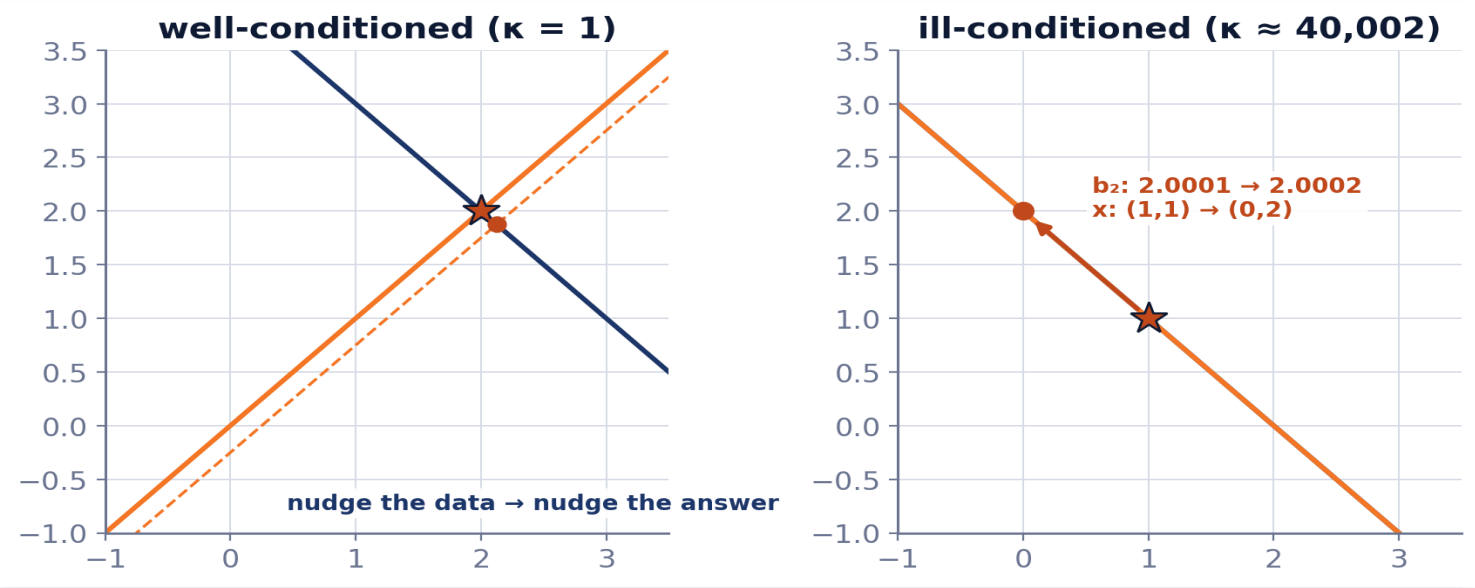
forward: $h \approx 1.5 \times 10^{-8} \rightarrow$ error 6×10^{-9}

central: $h \approx 5.8 \times 10^{-6} \rightarrow$ error 7×10^{-13}

rule of thumb: $h = \sqrt{\varepsilon}$ (forward), $h = \sqrt[3]{\varepsilon}$ (central)

The gradient-check ritual (used in EVERY lab from Week 5 on): compare your analytic gradient g with the central-difference estimate ĝ via $\|g - \hat{g}\| / (\|g\| + \|\hat{g}\|)$. Below 10^{-6}: correct. Above 10^{-3}: you have a bug — almost always a missing factor or a sign.

Conditioning: When a Tiny Change Moves the Answer



Rule of thumb: you lose about $\log_{10}\kappa$ digits. $\kappa \approx 4\cdot10^4$ costs ~4½ of your 16 digits; $\kappa \approx 10^{16}$ costs all of them. Geometrically the two equations are nearly parallel lines, so their intersection is barely determined. The same κ (of the Hessian) will set gradient descent's speed in Week 5 — conditioning is both an ACCURACY and a SPEED story.

WORKED EXAMPLE

A = [[1, 1], [1, 1.0001]]

$b = (2, 2.0001) \rightarrow x = (1, 1)$

$b = (2, 2.0002) \rightarrow x = (0, 2)$

input changed by 0.0035 %

output changed by 100 %

amplification ≈ 28,000×

$\kappa(A) = \|A\|\,\|A^{-1}\| = \lambda max/\lambda min \approx 40{,}002$

(κ is the worst-case amplification)

Lab 2: Recompute Every Number on These Slides

- **Norms & the nearest-server flip; inner products and angles.**
- Gradients by hand, then a reusable gradient checker (central differences).
- Hessians + eigenvalues: certify the bowl, expose the saddle, meet Rosenbrock's κ.
- Taylor models: reproduce the 3.60 / 9.86 / 9.37 verdict.
- Three definiteness tests; solve Qx = b; convexity by falsification.
- Numerical reality: cancellation, the V-curve, the jumping linear system.

THE ARC OF LAB 2

1. **COMPUTE —** every worked example, in NumPy, asserted
2. **CHECK —** build the gradient checker you will reuse all term
3. **BREAK —** make floating point and conditioning fail on purpose

```
def grad_check(f, grad, x, h=6e-6):           # the ritual, in 7 lines
    g, g_hat = grad(x), np.zeros_like(x)
    for i in range(x.size):
        e = np.zeros_like(x); e[i] = h        # perturb ONE coordinate
        g_hat[i] = (f(x + e) - f(x - e)) / (2 * h)   # central difference
    return np.linalg.norm(g - g_hat) / (np.linalg.norm(g) + np.linalg.norm(g_hat))
# < 1e-6: correct     > 1e-3: bug
```

HW1 builds on this lab: derive, implement, and gradient-check three functions of your choice.

The One-Page Toolkit

OBJECT	WHAT IT IS	WHAT YOU DO WITH IT	USED
Inner product / norms	$x^Ty = \|\|x\|\|\|\|y\|\|\cos\theta$; L_1, L_2, $L\infty$	descent test $\nabla f^Tp < 0$; the norm IS the objective	all
Gradient ∇f	vector of partials; $\perp$ contours	$\nabla f = 0$ at candidates; cookbook: $Qx - b$, $2A^T(Ax - b)$	Wks 5, 10
Hessian $\nabla^2 f$	symmetric matrix of 2nd partials	eigenvalues: all > 0 bowl · mixed = saddle; $\kappa = \lambda max/\lambda min$	Wk 6
Taylor model	$f + \nabla f^Tp + \frac{1}{2}p^T\nabla^2 f\, p$	errors ~p^2 and ~p^3 — models are LOCAL	Wks 5–6
Quadratic $\frac{1}{2}x^TQx - b^Tx$	PD $\Rightarrow$ unique min at $Qx = b$	tests: eigenvalues · leading minors · Cholesky	Wks 5–9
Convexity	chord test; tangents underestimate; $\nabla^2 f$ PSD	local = global; certify by blocks + 3 rules	Wks 3–9
Numerical reflexes	16 digits; κ amplifies errors	avoid cancellation · vectorize · gradient-check · watch κ	every lab

Photograph this slide. It is the legend for every other week's map.

WEEK 2 — TAKEAWAYS

1. **The gradient is a compass —** $-\nabla f$ is steepest descent; any p with $\nabla f^T p < 0$ goes downhill; $\nabla f = 0$ marks the candidates.

2. **Eigenvalues, not coefficients —** $x^2 + 3xy + 2y^2$ looks like a bowl and is a saddle — only the Hessian's spectrum tells the truth.

3. **Models are local —** the linear model promised 3.60, the truth was 9.37 — curvature decides how far a step can be trusted.

4. **Convexity is the watershed —** nine starts, one answer — versus a 5–4 split. Certify it by blocks and rules, not by hope.

5. **Respect the machine —** sixteen digits: rewrite cancelling formulas, vectorize, gradient-check everything, watch κ.

BEFORE NEXT WEEK

- Lab 2: recompute every number; keep grad_check().
- Read: Boyd & Vandenberghe §2.1–2.3, §3.1–3.2; TR: Pınar, Dışbükeylik ve Optimizasyon, Ders 1–2.
- **HW1: derive, implement, gradient-check three functions.**

NEXT WEEK

Linear Programming I: Geometry & the Simplex Method

Today's polyhedra become feasible regions — and the optimum turns out to live at a corner.